P.Vishwa Nandan(192473015)

TASK 4 – Design Classes, Objects and Constructors

Marks: 4

This task is about designing Java classes and deciding how objects are created using constructors.

The assignment asks us to explain:

1. Fields and methods of each class.
2. Default, parameterized and overloaded constructors.
3. Use of this keyword and constructor chaining.
4. Copy constructors.
5. Stack and heap memory.
6. Alternative approaches and complexity analysis. CO1 JAVA

1. Fields and Methods of Each Class

The Smart-Parking System uses the following classes.

1. ParkingFacility

Fields:

- slots – Stores all parking slots.
- totalFloors – Stores the number of floors.

Methods:

- allocateSlot() – Allocates an available slot.
- releaseSlot() – Releases an occupied slot.
- findAvailableSlot() – Searches for an empty slot.

2. ParkingFloor

Fields:

- floorNumber – Stores the floor number.
- slots – Stores the parking slots on that floor.

Methods:

- displayAvailableSlots() – Displays free slots.
- getSlot() – Returns a particular parking slot.

3. ParkingSlot

Fields:

- slotNumber – Stores the slot number.
- isOccupied – Checks whether the slot is occupied.
- vehicle – Stores the vehicle parked in the slot.

Methods:

- occupySlot() – Assigns a vehicle.
- releaseSlot() – Makes the slot available.

4. Vehicle

Fields:

- registrationNumber – Stores the vehicle registration number.
- vehicleType – Stores the vehicle type.

Methods:

- getRegistrationNumber() – Returns the registration number.
- displayDetails() – Displays vehicle information.

5. ParkingTicket

Fields:

- ticketNumber – Stores the unique ticket number.
- vehicle – Stores vehicle details.
- entryTime – Stores the vehicle entry time.
- floorNumber – Stores the allocated floor.
- slotNumber – Stores the allocated slot.

Methods:

- generateTicket() – Creates a parking ticket.
- displayTicket() – Displays ticket information.

6. ParkingPass

Fields:

- passNumber – Stores the pass number.
- validFrom – Stores the starting date.

- validTo – Stores the expiry date.

Methods:

- validatePass() – Checks whether the pass is valid.
- displayPass() – Displays pass information.

---

2. Default, Parameterized and Overloaded Constructors

A constructor is a special method in Java that is called automatically when an object is created.

There are three important constructor concepts.

A. Default Constructor

A default constructor does not accept any parameters.

Example:

```
class ParkingFacility {

    ParkingFacility() {
        System.out.println("Parking facility created");
    }
}
```

It is useful when creating an object with predefined default values.

B. Parameterized Constructor

A parameterized constructor accepts values when an object is created.

Example:

```
class Vehicle {

    String registrationNumber;
    String vehicleType;

    Vehicle(String regNo, String type) {
        registrationNumber = regNo;
```

```
        vehicleType = type;
    }
}
```

Object creation:

```
Vehicle v = new Vehicle("TN01AB1234", "Car");
```

It initializes the object with the given vehicle information.

C. Overloaded Constructor

Constructor overloading means creating multiple constructors in the same class with different parameter lists.

Example:

```
class ParkingTicket {

    String ticketNumber;
    String vehicleNumber;

    ParkingTicket(String ticketNo) {
        ticketNumber = ticketNo;
    }

    ParkingTicket(String ticketNo, String vehicleNo) {
        ticketNumber = ticketNo;
        vehicleNumber = vehicleNo;
    }
}
```

It allows objects to be created in different ways depending on the information available.

---

3. Use of this Keyword and Constructor Chaining

The this keyword refers to the current object.

It is used to distinguish instance variables from constructor parameters.

Example:

```
class Vehicle {

    String registrationNumber;

    Vehicle(String registrationNumber) {
        this.registrationNumber = registrationNumber;
    }
}
```

Here:

- this.registrationNumber refers to the class variable.
- registrationNumber refers to the constructor parameter.

Constructor Chaining

Constructor chaining means calling one constructor from another constructor using this().

Example:

```
class ParkingTicket {

    String ticketNumber;
    String vehicleNumber;

    ParkingTicket(String ticketNo) {
        this(ticketNo, "Unknown");
    }

    ParkingTicket(String ticketNo, String vehicleNo) {
        this.ticketNumber = ticketNo;
        this.vehicleNumber = vehicleNo;
    }
}
```

The first constructor calls the second constructor using this().

This avoids repeating initialization code and improves code reusability.

---

4. Copy Constructor

A copy constructor creates a new object using the information from an existing object.

Example:

```
class Vehicle {

   String registrationNumber;

   Vehicle(String regNo) {
      this.registrationNumber = regNo;
   }

   Vehicle(Vehicle v) {
      this.registrationNumber = v.registrationNumber;
   }
}
```

Object creation:

```
Vehicle v1 = new Vehicle("TN01AB1234");

Vehicle v2 = new Vehicle(v1);
```

Here, v2 receives the registration number of v1.

Is a copy constructor needed in the parking system?

A copy constructor is not necessary for the ParkingTicket class because every ticket must have a unique ticket number.

If a duplicate ticket is required, the system should generate a new ticket number instead of copying the original ticket number.

This prevents confusion between active parking tickets.

---

5. Object Creation and Memory Management (Stack and Heap)

Java uses stack and heap memory while creating and managing objects.

Stack Memory:

- Stores local variables and method execution information.
- Stores local reference variables in the usual conceptual model.
- Memory is managed automatically when method execution finishes.

Heap Memory:

- Stores objects created using the new keyword.
- Stores arrays and their elements.
- Objects remain available while they are reachable.

Example:

Vehicle v = new Vehicle("TN01AB1234");

Explanation:

1. Vehicle is the class name.
2. v is the reference variable.
3. new Vehicle() creates a Vehicle object in heap memory.
4. v refers to that object.

When the vehicle leaves and the system no longer needs the corresponding object, it can become eligible for garbage collection if no references remain.

Garbage collection automatically manages unused objects.

---

6. Alternative Concept and Complexity Analysis

In the original design, objects are created using overloaded constructors and constructor chaining.

Alternatively, objects can be created using default constructors followed by setter methods or through static factory methods.

A. Default Constructor with Setter Methods

In this approach, an object is created first, and values are assigned using setter methods.

Example:

class ParkingTicket {

```
    String ticketNumber;
    String vehicleNumber;

    void setTicketNumber(String ticketNo) {
        ticketNumber = ticketNo;
    }

    void setVehicleNumber(String vehicleNo) {
        vehicleNumber = vehicleNo;
    }
}
```

Object creation:

```
ParkingTicket t = new ParkingTicket();

t.setTicketNumber("T101");
t.setVehicleNumber("TN01AB1234");
```

The disadvantage is that the object may contain incomplete information until all setter methods are called.

B. Static Factory Method

A static factory method creates and returns an object through a static method.

Example:

```
class ParkingTicket {

    String ticketNumber;
    String vehicleNumber;

    private ParkingTicket(String ticketNo, String vehicleNo) {
        ticketNumber = ticketNo;
```

```
        vehicleNumber = vehicleNo;
    }

    static ParkingTicket createTicket(String ticketNo, String vehicleNo) {
        return new ParkingTicket(ticketNo, vehicleNo);
    }
}
```

Object creation:

```
ParkingTicket t = ParkingTicket.createTicket("T101", "TN01AB1234");
```

This approach provides a clear method for creating valid objects and hides the constructor from direct access.

Complexity Analysis

1. Constructor Approach:

- Time complexity per object: O(1).
- Time complexity for n objects: O(n).
- Space complexity: O(n) for storing n objects.

2. Default Constructor with Setters:

- Time complexity per object: O(1), assuming a fixed number of setters.
- Time complexity for n objects: O(n).
- Space complexity: O(n).
- Requires additional setter calls and validation.

3. Static Factory Method:

- Time complexity per object: O(1).
- Time complexity for n objects: O(n).
- Space complexity: O(n).

All three approaches have the same overall asymptotic creation complexity when each object has a fixed number of fields. However, constructors and static factory methods help ensure that objects are properly initialized.

7. Conclusion

Constructors are used to initialize objects in the Smart-Parking System. Parameterized and overloaded constructors allow objects to be created with different information. The this keyword and constructor chaining reduce repeated code. Objects are stored in heap memory and managed through references. Constructor-based object creation is preferred because it helps maintain valid and properly initialized objects.